

面向对象程序设计 (C++) 作业

作业号	单链表的创建和删除
班级	信电学院计算机
学号	
姓名	

表 1 作业信息表

一、题目内容

已经结构体变量类型：

```
struct Score
{
    int english;
    struct Score *next;
};
```

利用其创建单链表，输入 e 与 enew，在 english 值为 e 的结点之前插入 english 值为 enew 的新结点，若找不到 english 为 e 的结点，则将新结点插入到链表最后。

例 1（第 1、2、7 行为输入，其他行为输出）：

```
3
88 99 77
1:77
2:99
3:88
Please input scores:
77 89
1:89
2:77
3:99
4:88
```

例 2(第 1、2、6 行为输入，其他为输出)：

2

44 66

1:66

2:44

Please input scores:

55 88

1:66

2:44

3:88

main 函数或给定程序如下:

```
//前插法创建单链表
struct Score *createList(int n)
{
    int i;
    struct Score *head=NULL,*p;
    for(i=1; i<=n ;i++)
    {
        p=new struct Score;
        cin>>p->english;
        if(i==1) { head=p;p->next=NULL; }
        else { p->next=head;head=p; }
    }
    return head;
}

//显示链表内容
void showList(struct Score *head)
{
    struct Score *p=head;
    int i=1;
    if(p==NULL) { cout<<"Empty!"<<endl; return;}
    while(p!=NULL)
    { cout<<i<<": "<<p->english<<endl; p=p->next; i++; }
}

//主函数
int main()
{
    struct Score *h;
    int n,e,enew;
    cin>>n;
    h=createList(n);
    showList(h);
    cout<<"Please input scores:"<<endl;
    cin>>e>>enew;
```

```

    h=insertBefore(h,e,ew);
    showList(h);
    return 0;
}

```

二、源程序

```

#include <iostream>
using namespace std;

// 结构体定义
struct Score {
    int english;
    struct Score *next;
};

// 前插法创建单链表
struct Score *createList(int n) {
    int i;
    struct Score *head = NULL, *p;
    for (i = 1; i <= n; i++) {
        p = new struct Score;
        cin >> p->english;
        if (i == 1) {
            head = p;
            p->next = NULL;
        } else {
            p->next = head;
            head = p;
        }
    }
    return head;
}

// 显示链表内容
void showList(struct Score *head) {
    struct Score *p = head;
    int i = 1;
    if (p == NULL) {
        cout << "Empty!" << endl;
        return;
    }
    while (p != NULL) {
        cout << i << ":" << p->english << endl;
        p = p->next;
    }
}

```

```

        i++;
    }
}

// 关键: 在english值为e的结点之前插入enew
struct Score *insertBefore(struct Score *head, int e, int enew) {
    struct Score *p = head;
    struct Score *q = NULL; // 用来记录p的前一个结点

    struct Score *newnode = new struct Score;
    newnode->english = enew;
    newnode->next = NULL;
    while (p != NULL)
    {
        if (p->english == e)
        {
            if (q == NULL)
            {
                // 说明要插到头结点之前
                newnode->next = head;
                head = newnode;
            }
            else
            {
                // 说明要插到p之前
                q->next = newnode;
                newnode->next = p;
            }
            return head;
        }
        q = p;
        p = p->next;
    }
    // 如果找不到, 插到末尾
    if (p == NULL)
    {
        q->next = newnode;
    }
    else
    {
        // 原链表为空的情况
        head = newnode;
    }
    return head;
}

// 主函数

```

```
int main() {  
    struct Score *h;  
    int n, e, enew;  
    cin >> n;  
    h = createList(n);  
    showList(h);  
    cout << "Please input scores:" << endl;  
    cin >> e >> enew;  
    h = insertBefore(h, e, enew);  
    showList(h);  
    return 0;  
}
```

三、insertBefore 函数详解

```
// 关键：在english值为e的结点之前插入enew
struct Score *insertBefore(struct Score *head, int e, int enew) {
    struct Score *p = head;
    struct Score *q = NULL; // 用来记录p的前一个结点

    struct Score *newnode = new struct Score;
    newnode->english = enew;
    newnode->next = NULL;
    while (p != NULL)
    {
        if (p->english == e)
        {
            if (q == NULL)
            {
                // 说明要插到头结点之前
                newnode->next = head;
                head = newnode;
            }
            else
            {
                // 说明要插到p之前
                q->next = newnode;
                newnode->next = p;
            }
            return head;
        }
        q = p;
        p = p->next;
    }
    // 如果找不到，插到末尾
    if (p == NULL)
    {
        q->next = newnode;
    }
    else
    {
        // 原链表为空的情况
        head = newnode;
    }
    return head;
}
```

3.1 函数定义

```
struct Score *insertBefore(struct Score *head, int e, int enew)
```

功能:

- 在链表中，找到第一个 `english == e` 的节点，在它前面插入一个新节点，新节点的 `english == enew`。

参数:

- `head`: 指向链表的头节点
- `e`: 要找的分数
- `enew`: 要插入的新分数

返回值:

- 插入后新的链表头 (`head` 可能会改变)

3.2 定义两个指针，创建新节点

```
struct Score *p = head;  
struct Score *pre = NULL;
```

- `p`: 用于遍历链表的当前节点
- `pre`: 用于记录 `p` 的前一个节点，因为插入操作需要知道前面是谁！

```
struct Score *newNode = new struct Score;  
newNode->english = enew;  
newNode->next = NULL;
```

- 创建一个新的节点 `newNode`
- 设置 `newNode->english = enew`
- 初始化它的 `next` 指针为 `NULL`
- 为什么？因为还不知道插到哪里，先空着。

3.3 开始遍历链表，寻找目标节点

```
while (p != NULL)
```

- 从头开始，一直走到链表末尾
- `p` 指向当前考察的节点

3.4 如果找到了目标节点

```
if (p->english == e)
```

- 说明当前节点 p 就是 $\text{english} == e$ 的节点

3.5 分情况插入

3.5.1 如果是插在头节点前

```
if (pre == NULL)
```

- 如果 pre 是空指针，说明 p 就是头节点。
- 也就是说，目标元素在最前面，那么新节点应该变成新的头。

```
newNode->next = head;  
head = newNode;
```

- 让新节点 newNode 指向当前的头节点（也就是 p ）
- 更新 head ，指向 newNode

插入前： $\text{head} \rightarrow [77] \rightarrow [99] \rightarrow [88] \rightarrow \text{NULL}$

插入后： $\text{head} \rightarrow [89] \rightarrow [77] \rightarrow [99] \rightarrow [88] \rightarrow \text{NULL}$

3.5.2 如果是插在中间（不是头节点）

```
else {  
    pre->next = newNode;  
    newNode->next = p;  
}
```

- 否则，目标节点 p 不是头节点，在 pre 和 p 之间插入 newNode
- 两步：
 - $\text{pre->next} = \text{newNode}$; 先让前一个节点指向新节点
 - $\text{newNode->next} = p$; 新节点再指向当前节点

插入前： $\text{pre} \rightarrow [A] \rightarrow p([77]) \rightarrow [99] \rightarrow \text{NULL}$

插入后： $\text{pre} \rightarrow [A] \rightarrow \text{newNode}([89]) \rightarrow p([77]) \rightarrow [99] \rightarrow \text{NULL}$

3.6 插入完成，返回新的头指针

```
return head;
```

- 插入完毕，直接返回新的头结点 head
- 注意：如果是插在头部，head 已经更新了

3.7 如果整个链表都找不到 e，插到链表最后

遍历继续走：

```
pre = p;  
p = p->next;
```

- pre 指向当前节点 p
- p 往后走一格
- 最后遍历到 p == NULL，跳出循环

插到尾巴上：

```
if (pre != NULL)  
{  
    pre->next = newNode;  
}
```

- 说明链表不是空的
- 最后一个节点（pre）的 next 指向新节点

特别注意：链表为空的情况！

```
else  
{  
    head = newNode;  
}
```

- 如果原链表是空链表（head == NULL，pre == NULL）
- 那么新节点自己成为新的头结点！

3.8 最后，返回新链表头

```
return head;
```

表 2 链表插入处理方法总结

场景	处理方法	关键变化
找到 e 且 p 是头节点	newNode 接 head, head 指向 newNode	更新 head
找到 e 且 p 不是头节点	pre 接 newNode, newNode 接 p	中间插入
没找到 e	pre 接 newNode (尾插) 或 head=newNode (空链表)	尾插
返回值	返回更新后的 head	保证主程序拿到新链表

总结一下就是：“找到了就前插，找不到就尾插，链表为空就自己做头!”